TO-BE Reto 2 sobre TO-BE Reto 1 — mismo lienzo, dos capas reales de draw.io
Apague la capa "TO-BE Reto 2" en el panel Capas y queda solo el Reto 1 original; enciendala y el patron nuevo aparece exactamente encima de cada clase que toca.

Leyenda

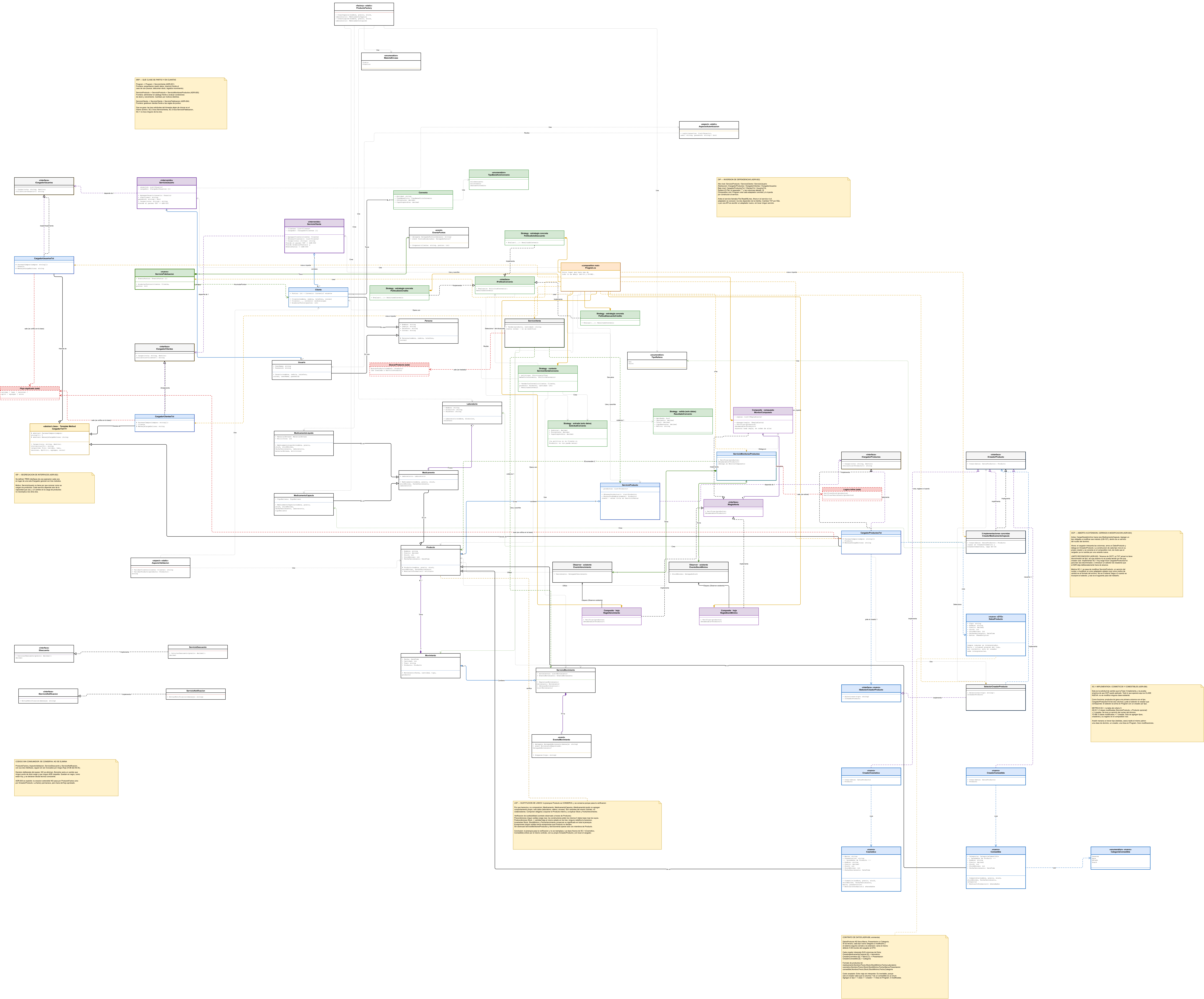
→	Construye (new, desde el Composition Root)	🟧	Composition Root (Program.cs)
—▷	Herencia (es un)	☐	AS-IS — se conserva sin cambios
- - ▷	Realizacion (implementa)	🟦	AS-IS — misma clase, cambia el interior
◆→	Composicion (rombo relleno en el dueño)	🟥	Sale (logica que se elimina o se unifica)
···>	Dependencia de uso	🟩	Nuevo — Strategy (P-02 / SC-3)
—	Asociacion (multiplicidad)	🟪	Nuevo — Composite (P-03)
→	Flujo / evento	🟨	Nuevo — Template Method (P-04)
- - >	Sale (logica que se elimina o se unifica)		

Leyenda — convencion de color por principio aplicado

- ☐ CONSERVADO del diseño original — misma caja, mismos miembros, misma posicion que en el AS-IS
- 🟩 SRP — Responsabilidad unica
- 🟦 OCP — Abierto/cerrado
- 🟪 DIP — Inversion de dependencias
- 🟧 ISP — Segregacion de interfaces
- 🟨 LSP — jerarquia CONSERVADA y verificada (ver nota)

«nuevo» = clase que no existia · «intervenido» = clase del AS-IS cuyo contenido cambia · sin estereotipo = identica al AS-IS

SolucionFarmacia — Diagrama de clases TO-BE (Fase 3)
Construido sobre el diagrama AS-IS: cada clase conservada esta en la MISMA posicion y con los MISMOS miembros, en NEGRO.
EN COLOR unicamente lo intervenido, con un color por principio aplicado · trazado a los ADR-001 a ADR-005


«factory» «static»
ProductoFactory
+ CrearCapsula(nombre, precio, stock, laboratorio): MedicamentoCapsula
+ CrearLiquido(nombre, precio, stock, laboratorio): MedicamentoLiquido
Usa
«enumeration»
MaterialEnvase
Vidrio
Plastico
«aspect» «static»
AspectoAutenticacion
+ Login(usuarios: List<Usuario>, user: string, password: string): bool
Usa
Recibe
«interface»
ICargadorUsuarios
+ Cargar(ruta: string, destino: ICollection<Usuario>): string
depende de 1
«intervenido»
ServicioUsuario
- usuarios: List<Usuario>
- cargador: ICargadorUsuarios [1]
+ AgregarUsuario(usuario: Usuario)
+ Login(user: string, password: string): bool
+ Cargar(ruta: string): string
Pierde el parseo TXT -> ADR-002
implemImplementa
CargadorUsuariosTxt
# ParsearCampos(campos: string[]): Usuario
# MensajeCargaExitosa: string
«enumeration»
TipoBeneficioConvenio
SoloDescuento
SoloCredito
DescuentoCredito
Usa
Convenio
+ Entidad: string
+ TipoBeneficio: TipoBeneficioConvenio
+ Porcentaje: decimal
+ CupoDisponible: decimal
0..1
«intervenido»
ServicioCliente
- clientes: List<Cliente>
- cargador: ICargadorClientes [1]
+ AgregarCliente(cliente: Cliente)
+ ObtenerClientes(): List<Cliente>
+ Cargar(ruta: string): string
Pierde el parseo TXT -> ADR-002
Pierde AcumularPuntos() y EventoPuntos -> ADR-004
«event»
EventoPuntos
+ delegate DelegadoPuntos(mensaje: string)
+ event PuntosAcumulados: DelegadoPuntos?
+ Disparar(cliente: string, puntos: int)
Strategy · estrategia concreta
PoliticaSoloDescuento
+ Evaluar(...): ResultadoConvenio
Implementa
«composition root»
Program.cs
Unico lugar que hace new de todo lo de abajo (AS-IS y TO-BE).
crea e inyecta
Usa y suscribe
Tiene
«nuevo»
ServicioFidelizacion
+ EventoPuntos: EventoPuntos [1]
+ AcumularPuntos(cliente: Cliente, puntos: int)
Contiene
AcumularPuntos
contiene
Cliente
+ Puntos: int + Convenio: Convenio? «nuevo»
+ Cliente(nombre, cedula, telefono, correo)
+ Cliente(..., convenio) «sobrecarga»
+ AcumularPuntos(puntos: int)
Strategy · estrategia concreta
PoliticaSoloCredito
+ Evaluar(...): ResultadoConvenio
Implementa
«interface»
IPoliticaConvenio
+ Evaluar(s: SolicitudConvenio): ResultadoConvenio
crea
Strategy · estrategia concreta
PoliticaDescuentoCredito
+ Evaluar(...): ResultadoConvenio
Opera con
Persona
+ Nombre: string
+ Cedula: string
+ Telefono: string
+ Correo: string
# Persona(nombre, cedula, telefono, correo)
Es una
ServicioVenta
+ Vender(producto, cantidad): string
(venta normal — no se modifica)
Selecciona 1 del diccionario
Recibe
«interface»
ICargadorClientes
+ Cargar(ruta: string, destino: ICollection<Cliente>): string
Usuario
+ UserName: string
+ Password: string
+ Usuario(nombre, cedula, telefono, correo, userName, password)
BuscarProducto (sale)
BuscarProducto(nombre): Producto?
(se traslada a ServicioProducto)
sale (se traslada)
«enumeration»
TipoRelleno
Gel
Polvo
Strategy · contexto
ServicioVentaConvenio
- politicas: IDictionary<TipoBeneficioConvenio, IPoliticaConvenio>
+ VenderConConvenio(cliente: Cliente, producto: Producto, cantidad: int): ResultadoConvenio
Devuelve
Crea
sale (se unifica en la base)
Hereda de
iImplementa
Flujo duplicado (sale)
validar / leer / recorrer / split / agregar / error
CargadorClientesTxt
# ParsearCampos(campos: string[]): Cliente
# MensajeCargaExitosa: string
«abstract class» · Template Method
CargadorTxt<T>
# abstract ParsearCampos(campos: string[]): T
# abstract MensajeCargaExitosa: string
+ Cargar(ruta: string, destino: ICollection<T>): string
(algoritmo fijo: validar, leer, recorrer, Split(;), agregar, error)
Laboratorio
+ Nombre: string
+ Direccion: string
+ Telefono: string
+ Laboratorio(nombre, direccion, telefono)
Strategy · entrada (solo datos)
SolicitudConvenio
+ Subtotal: decimal
+ Porcentaje: decimal
+ CupoDisponible: decimal
(la politica no ve Cliente ni Producto: no los puede mutar)
Strategy · salida (solo datos)
ResultadoConvenio
+ Aprobado: bool
+ Descuento: decimal
+ Total: decimal
+ CupoRestante: decimal
+ Motivo: string
colabora 1
Usa y suscribe
Composite · compuesto
MonitorCompuesto
- reglas: List<IReglaAlerta>
+ Agregar(regla: IReglaAlerta)
+ Verificar(productos: IEnumerable<Producto>)
(ejecuta cada regla, en orden de alta)
Delega en
ServicioMonitoreoProductos
+ Verificar(productos: IEnumerable<Producto>)
(delega en MonitorCompuesto)
IEnumerable 0..*
Posee
MedicamentoLiquido
+ MaterialEnvase: MaterialEnvase
+ Mililitros: int
+ MedicamentoLiquido(nombre, precio, stock, stockMinimo, fechaVencimiento, laboratorio, materialEnvase, mililitros)
Es un
Medicamento
+ Laboratorio: Laboratorio
+ Medicamento(nombre, precio, stock, stockMinimo, fechaVencimiento, laboratorio)
MedicamentoCapsula
+ TipoRelleno: TipoRelleno
+ MedicamentoCapsula(nombre, precio, stock, stockMinimo, fechaVencimiento, laboratorio, tipoRelleno)
ServicioProducto
- productos: List<Producto>
+ ObtenerProductos(): List<Producto>
+ BuscarPorNombre(nombre): Producto?
(nuevo — antes vivia en ServicioVenta)
contiene
Implementa
Compone
depende de 1
«interface»
IReglaAlerta
+ Verificar(productos: IEnumerable<Producto>)
sale (se extrae)
«interface»
ICargadorProductos
+ Cargar(ruta: string, destino: ICollection<Producto>): string
Logica inline (sale)
VerificarStock(productos)
VerificarVencimiento(productos)
«interface»
ICreadorProducto
+ Crear(datos: DatosProducto): Producto
crea, registra e inyecta
CargadorProductosTxt
# ParsearCampos(campos: string[]): Producto
# MensajeCargaExitosa: string
3 implementaciones concretas
CreadorMedicamentoCapsula
+ Crear(datos: DatosProducto): Producto
(igual en CreadorCosmetico y CreadorComestible, capa AS-IS)
resuelve 1
Selecciona
«aspect» «static»
AspectoValidacion
+ ValidarCliente(cliente: Cliente): string
+ ValidarProducto(producto: Producto): string
Producto
+ Nombre: string
+ Precio: decimal
+ Stock: int
+ StockMinimo: int
+ FechaVencimiento: DateTime
# Producto(nombre, precio, stock, stockMinimo, fechaVencimiento)
+ MostrarInformacion() «virtual»
Observer · existente
EventoVencimiento
+ Vencimiento: DelegadoVencimiento
Observer · existente
EventoStockMinimo
+ StockMinimo: DelegadoStock
Utiliza
Dispara (Observer existente)
Composite · hoja
ReglaVencimiento
+ Verificar(productos: IEnumerable<Producto>)
Composite · hoja
ReglaStockMinimo
+ Verificar(productos: IEnumerable<Producto>)
«nuevo» «DTO»
DatosProducto
+ Tipo: string
+ Nombre: string
+ Precio: decimal
+ Stock: int
+ StockMinimo: int
+ FechaVencimiento: DateTime
+ Extra: IReadOnlyList
Campos comunes ya interpretados. Extra = columnas propias del tipo, sin convertir: solo el creador sabe interpretarlas.
pide el creador 1
Usa
«interface»
IDescuento
+ CalcularDescuento(precio: decimal): decimal
ServicioDescuento
+ CalcularDescuento(precio: decimal): decimal
Implementa
Tiene
Movimiento
+ Fecha: DateTime
+ Cantidad: int
+ Tipo: string
+ Producto: Producto
+ Movimiento(fecha, cantidad, tipo, producto)
Contiene
ServicioMovimiento
- movimientos: List<Movimiento>
+ EventoMovimiento: EventoMovimiento
+ RegistrarMovimiento(movimiento: Movimiento)
+ ObtenerMovimientos(): List<Movimiento>
verifica
«interface» «nuevo»
ISelectorCreadorProducto
+ Seleccionar(tipo: string): ICreadorProducto
implementa
SelectorCreadorProducto
+ Seleccionar(tipo: string): ICreadorProducto
«interface»
IServicioNotificacion
+ EnviarNotificacion(mensaje: string)
ServicioNotificacion
+ EnviarNotificacion(mensaje: string)
«event»
EventoMovimiento
+ delegate DelegadoMovimiento(mensaje: string)
+ event MovimientoRegistrado: DelegadoMovimiento?
+ Disparar(tipo: string)
«nuevo»
CreadorCosmetico
+ Crear(datos: DatosProducto): Producto
«nuevo»
CreadorComestible
+ Crear(datos: DatosProducto): Producto
crea
«nuevo»
Cosmetico
+ Marca: string
+ Presentacion: string
--- heredados de Producto ---
+ Nombre: string
+ Precio: decimal
+ Stock: int
+ StockMinimo: int
+ FechaVencimiento: DateTime
+ Cosmetico(nombre, precio, stock, stockMinimo, fechaVencimiento, marca, presentacion)
+ MostrarInformacion() «heredado»
«nuevo»
Comestible
+ Categoria: CategoriaComestible
--- heredados de Producto ---
+ Nombre: string
+ Precio: decimal
+ Stock: int
+ StockMinimo: int
+ FechaVencimiento: DateTime
+ Comestible(nombre, precio, stock, stockMinimo, fechaVencimiento, categoria)
+ MostrarInformacion() «heredado»
usa
«enumeration» «nuevo»
CategoriaComestible
Gaseosa
Agua
Helado
Snack
Es un

SRP — QUE CLASE SE PARTIO Y EN CUANTAS

Program -> Program + ServicioVenta (ADR-001)
Frontera: presentacion (pedir datos, imprimir) frente al caso de uso (buscar, descontar stock, registrar movimiento).

ServicioProducto -> ServicioProducto + ServicioMonitoreoProductos (ADR-005)
Frontera: administrar el catalogo frente a evaluar condiciones de stock y vencimiento. Cambian por motivos distintos.

ServicioCliente -> ServicioCliente + ServicioFidelizacion (ADR-004)
Frontera: gestionar clientes frente a las reglas de puntos.

Que se gana: las tres solicitudes del trimestre dejan de chocar en el mismo archivo. SC-2 toca ServicioVenta, SC-3 toca ServicioFidelizacion, SC-1 no toca ninguno de los dos.

DIP — INVERSION DE DEPENDENCIAS (ADR-002)

Alto nivel: ServicioProducto / ServicioCliente / ServicioUsuario
Abstraccion: ICargadorProductos / ICargadorClientes / ICargadorUsuarios
Bajo nivel: CargadorProductosTxt / ClientesTxt / UsuariosTxt, System.IO.File, el separador ";" y las columnas datos[0..5]
Composition root: Program crea cada adaptador concreto y lo inyecta por constructor al servicio.

Antes el servicio llamaba File.ReadAllLines. Ahora ni el servicio ni el adaptador se conocen: los dos dependen de la interfaz. Cambiar TXT por SQL o por una API es escribir un adaptador nuevo, sin tocar ningun servicio.

ISP — SEGREGACION DE INTERFACES (ADR-002)

Se definen TRES interfaces de una operacion cada una en lugar de una sola ICargador general con tres metodos.

Motivo: ServicioUsuario no tiene por que conocer como se cargan los productos. Cada servicio depende solo de la capacidad que usa, y un cambio en la carga de productos no recompila a los otros dos.

OCP — ABIERTO A EXTENSION, CERRADO A MODIFICACION (ADR-003)

Antes: CargarDesdeArchivo hacia new MedicamentoCapsula. Agregar un tipo obligaba a modificar ese metodo (L99-107), dentro de un servicio del nucleo del dominio.

Ahora: el cargador interpreta las columnas, arma un DatosProducto y delega en ICreadorProducto. La construccion de cada tipo vive en su propio creador y se conecta en el composition root, de modo que el cargador ya no cambia por una variante nueva.

LIMITE RECONOCIDO (ADR-003, "Alcance de OCP"): el TXT actual no tiene discriminador de tipo, asi que todavia no se puede decidir por fila que creador usar. Implementar SC-1 hoy exige tocar CargadorProductosTxt para leer ese discriminador, o introducir un selector de creadores que el ADR deja deliberadamente fuera de alcance.

Metrica SC-1: se pasa de modificar ServicioProducto, un servicio del nucleo, a modificar un unico adaptador aislado cuyo unico motivo de cambio es el formato del archivo. No es 0 todavia: llega a 0 cuando se incorpore el selector, y ese es el siguiente paso del rediseño.

SC-1 IMPLEMENTADA: COSMETICOS Y COMESTIBLES (ADR-006)

Esta es la solicitud de cambio que la Fase 4 implementa, y la prueba empirica de que OCP quedo aplicado. Todo lo que aparece aqui es CLASE NUEVA: no se modifica ninguna clase existente.

Como funciona: productos.txt gana una primera columna con el tipo. CargadorProductosTxt lee esa columna y pide al selector el creador que corresponde. El selector se arma en Program con un creador por tipo.

METRICA SC-1, la tabla del criterio 5:
AS-IS 1-2 clases modificadas (ServicioProducto, y Producto opcional) + 2 creadas. Se toca un servicio del nucleo del dominio.
TO-BE 0 clases modificadas + 7 creadas. Solo se agregan tipos, creadores y su registro en el composition root.

Anadir manana un tercer tipo (bebidas, aseo) repite el mismo patron: una clase de dominio, un creador, una linea en Program. Cero modificaciones.

LSP — SUSTITUCION DE LISKOV: la jerarquia Producto se CONSERVA, y se conserva porque pasa la verificacion

Por que herencia y no composicion: Medicamento, MedicamentoCapsula y MedicamentoLiquido no agregan comportamiento propio, solo datos (laboratorio, relleno, envase). Son variantes del mismo contrato, no colaboradores. Componer obligaria a exponer el Producto interno y a duplicar Stock y FechaVencimiento.

Verificacion de sustituibilidad (contrato observado a traves de Producto):
Precondiciones ningun subtipo exige mas: los constructores piden los mismos 5 datos base mas los suyos.
Postcondiciones Stock -= cantidad deja el mismo estado en los tres; ninguno redefine la transicion.
Invariantes Stock, StockMinimo y FechaVencimiento conservan su significado en toda la jerarquia.
Excepciones ningun subtipo lanza excepciones que Producto no declare.
Sin downcast ServicioMonitoreoProductos y ServicioVenta operan solo con miembros de Producto.

Conclusion: la jerarquia pasa la verificacion y no se reemplaza. Los tipos futuros de SC-1 (Cosmetico, Comestible) entran por el mismo contrato, con su propio ICreadorProducto y sin tocar el cargador.

CODIGO SIN CONSUMIDOR: SE CONSERVA, NO SE ELIMINA

ProductoFactory, AspectoValidacion, ServicioDescuento y ServicioNotificacion, con sus dos interfaces, siguen sin ser invocados por ningun flujo (H-08 del AS-IS).

Decision deliberada del equipo: NO se eliminan. Borrarlos seria un cambio que ningun punto de dolor exige y que ningun ADR respalda. Quedan en negro, como estan hoy, y se declaran deuda tecnica consciente.

ADR-003 es explicito: la creacion extensible NO pasa por ProductoFactory sino por ICreadorProducto. La factory permanece, pero fuera del flujo aprobado.

CONTRATO DE DATOS (ADR-006, enmienda)

DatosProducto NO lleva Marca, Presentacion ni Categoria.
Si los llevara, cada tipo nuevo obligaria a modificarlo y el sistema seguiria cerrado a la extension: seria el mismo defecto H-06 movido del cargador al DTO.

Cada creador interpreta SUS columnas de Extra:
CreadorMedicamentoCapsula [0] -> laboratorio
CreadorCosmetico [0] -> Marca [1] -> Presentacion
CreadorComestible [0] -> Categoria

Formato de productos.txt:
medicamento;Nombre;Precio;Stock;StockMinimo;Fecha;Laboratorio
cosmetico;Nombre;Precio;Stock;StockMinimo;Fecha;Marca;Presentacion
comestible;Nombre;Precio;Stock;StockMinimo;Fecha;Categoria

Costo aceptado: Extra viaja sin interpretar. Es inevitable, porque solo el creador sabe que la columna 7 de un comestible es un enum.
Agregar un tipo = 1 clase + 1 creador + 1 linea en Program. 0 modificadas.